

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-index\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a2d370d8d417de881.jsonl`  
- SHA-256: `5faad054bd23f26ffbb8054d94f87313f55f3bf7656016cfc644848b00957345`  
- Source modified: `2026-06-25T17:00:50+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_c81dbe7a-7ff`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T16:57:11.751Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [NIT] No happy-path test for `_maybe_sweep_gemini_orphans` success/log path
Location: `tests/test_provider_gemini.py:195-207`
Detail: The only wiring test that reaches `provider_registry.get`` (`test_maybe_sweep_runs_and_swallows_errors`) does so via an exception side\_effect, so the success branch of `_maybe_sweep_gemini_orphans` (`backend/main.py:1204-1206`: positive ``deleted`` ? `logger.info 'reclaimed %d remote file(s)'`) is never executed. The ``if deleted:`` log branch is unverified.
Suggested fix: Add a test where ``cleanup_orphaned_files.return_value = 2`` and assert it was called once (and optionally that the provider was built with ``api_key`/`model_name`` from config); cheap and closes the happy-path wiring gap.

Return `isReal` + `adjustedSeverity` (`INVALID` if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:57:14.744Z)

I'll verify this finding by examining the actual code in the PR.

3. user (2026-06-25T16:57:16.392Z)

o1-p11/gemini-remote-file-gc
f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep

4. user (2026-06-25T16:57:17.127Z)

```
backend/main.py          | 25 ++++++++  
backend/providers/gemini.py | 43 ++++++++  
tests/test_provider_gemini.py | 104 ++++++++  
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:57:19.599Z)

Let me look at the actual code in `main.py`, `gemini.py`, and the test file.

6. user (2026-06-25T16:57:20.822Z)

```
diff --git a/backend/main.py b/backend/main.py  
index 5561cee..ea5a875 100644
```

```

--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure # standardized error/result contract
    from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
    sys.exit(1)

+def _maybe_sweep_gemini_orphans(config) -> None:
+    """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
    tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+def main():
+    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
+    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
    if swept:
        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
+    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
    single-shot.
    if args.video_path and not args.server and not args.app:
        print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
    import logging
    import os
    import time
+from datetime import datetime, timedelta, timezone
    from typing import Tuple

    from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

```

```

+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+         live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
+ deleted, grace_sec)
+     return deleted
+
+ def analyze_video(self, video_path: str, prompt: str,
+                 chunk_duration: float = 0.0,
+                 label: str = "") -> Tuple[list, dict]:

```

7. user (2026-06-25T16:57:21.079Z)

```

diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
     provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
     segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
     assert len(segments) == 0
+
+
+
+## --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name

```

```
+ f.mime_type = mime
+ f.create_time = create_time_iso
+ f.state = MagicMock()
+ f.state.name = state
+ return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),          # non-video -> kept
+        _file_mock("notts", "video/mp4", None),            # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises, second succeeds ? _safe_delete swallows it.
+        provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1 # only the second file counted
+
+
+# --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
```

```

+         lambda *a, **k: called.append(k) or MagicMock())
+ monkeypatch.setenv("GEMINI_API_KEY", "k")
+ main_mod._maybe_sweep_gemini_orphans(cfg)
+ assert called == [] # provider never instantiated
+
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
+    """No GEMINI_API_KEY -> no sweep (can't auth)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+        lambda *a, **k: called.append(k) or MagicMock())
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+    """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
+startup)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    cfg.ai_model = "gemini-2.5-flash"
+    provider = MagicMock()
+    provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+    monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    # Must not raise.
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    provider.cleanup_orphaned_files.assert_called_once()

```

8. assistant (2026-06-25T16:57:24.159Z)

Now I have the full picture. Let me verify the exact line numbers and confirm the claim about which tests reach `provider\_registry.get`.

9. user (2026-06-25T16:57:25.907Z)

```

1188 def _maybe_sweep_gemini_orphans(config) -> None:
1189     """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
1190     A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1191     a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
1192     try:
1193         provider_name = getattr(config, "ai_provider", "")
1194         if provider_name != "gemini":
1195             return
1196         api_key = os.environ.get("GEMINI_API_KEY")
1197         if not api_key:
1198             return
1199         provider = provider_registry.get("gemini", api_key=api_key,
model_name=config.ai_model)
1200         deleted = provider.cleanup_orphaned_files()
1201         if deleted:
1202             logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).",
deleted)
1203     except Exception as e:
1204         logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1205
1206

```

```

1207     def main():
1208         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
1209         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")

```

10. user (2026-06-25T16:57:26.214Z)

```

155     ]
156     with patch("backend.providers.gemini.genai"):
157         provider = GeminiProvider(api_key="k", model_name="m")
158         provider._client = MagicMock()
159         provider._client.files.list.return_value = iter(files)
160         # First delete raises, second succeeds ? _safe_delete swallows it.
161         provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162         deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163         assert deleted == 1 # only the second file counted
164
165
166
167     # --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == [] # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()
186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
197         import backend.main as main_mod
198         cfg = MagicMock()
199         cfg.ai_provider = "gemini"
200         cfg.ai_model = "gemini-2.5-flash"
201         provider = MagicMock()
202         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
203         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
204         monkeypatch.setenv("GEMINI_API_KEY", "k")
205         # Must not raise.
206         main_mod._maybe_sweep_gemini_orphans(cfg)
207         provider.cleanup_orphaned_files.assert_called_once()

```

11. assistant (2026-06-25T16:57:36.668Z)

Now let me carefully analyze the finding's accuracy.

The finding claims the ``if deleted:`` log branch in ``_maybe_sweep_gemini_orphans`` (`main.py:1201-1202`) is never executed by any test. Let me trace each of the three wiring tests:

1. ``test_maybe_sweep_skips_non_gemini_provider`` ? ``ai_provider="openai"``, returns early at line 1194-1195. Never reaches ``provider_registry.get`` or the ``deleted`` log.
2. ``test_maybe_sweep_skips_when_no_api_key`` ? no key, returns early at line 1197-1198. Never reaches the log branch.
3. ``test_maybe_sweep_runs_and_swallows_errors`` ? provider built, but ``cleanup_orphaned_files.side_effect = RuntimeError`` raises at line 1200, so execution jumps to the ``except`` at line 1203. The ``if deleted:`` log branch at line 1201-1202 is never reached.

The finding is factually correct: no test exercises the success path where ``deleted`` is a positive integer and the ``logger.info("[STARTUP] ... reclaimed %d remote file(s)")`` branch executes. The minor wording slip in the finding ("the only wiring test that reaches ``provider_registry.get``") is accurate ? only test #3 reaches ``get``, and it does so via an exception `side_effect`, so the post-``get`` success path including the log branch is untested.

Let me verify the assessment about severity context ? whether the provider-level method is well-covered (it is, by ``test_cleanup_orphaned_files_deletes_stale_videos_only``), which means only the thin `main.py` wiring's happy path is the gap.

The provider method ``cleanup_orphaned_files`` itself is thoroughly tested. The untested code is just the ``if deleted:`` log line in the wiring function plus the ``provider.cleanup_orphaned_files()`` return-value flow on the success path. This is a genuine but very minor test-coverage gap ? exactly NIT-level.

12. user (2026-06-25T17:00:45.490Z)

Structured output provided successfully

13. assistant (2026-06-25T17:00:50.383Z)

Verified the finding against the real PR #380 code.

The finding is real and correctly rated NIT. The success/log branch of ``_maybe_sweep_gemini_orphans`` (`backend/main.py:1201-1202`, the ``if deleted: logger.info(...reclaimed...)`` line) is genuinely never executed by any test ? the only wiring test that reaches ``provider_registry.get`` (``test_maybe_sweep_runs_and_swallows_errors``, `tests/test_provider_gemini.py:195-207`) sets ``cleanup_orphaned_files.side_effect = RuntimeError``, so line 1200 raises and control jumps straight to the ``except``. The other two wiring tests return early before ``get``. The underlying provider method is well covered, so this is a minor cosmetic-wiring coverage gap, not a correctness issue.